

Metody Obliczeniowe w Nauce i Technice

Układy równań liniowych - metoda Jacobiego

Adrianna Grobel

25.05.2026

1. Dane techniczne

- System operacyjny - macOS
- Procesor - Apple M3
- Pamięć RAM - 16 GB
- Język - Python 3.13.5
- Środowisko - PyCharm
- Precyzja obliczeń - typ float64 z biblioteki NumPy

2. Wprowadzenie

Celem ćwiczenia była numeryczna implementacja i analiza zbieżności iteracyjnej metody Jacobiego, służącej do rozwiązywania układów równań liniowych postaci $Ax = b$.

W ramach zadania rozpatrywane były układy o macierzy współczynników zdefiniowanej następująco:

$$A = \begin{cases} a_{ii} = 7 \\ a_{ij} = \frac{0.5}{n-i-j+0.5} \end{cases} \quad \text{dla } i \neq j \quad \text{dla } i, j = 1, \dots, n$$

Głównym założeniem eksperymentu była analiza, w jaki sposób dobrane parametry – rozmiar macierzy, wybór wektora startowego, założona tolerancja błędu oraz zastosowane kryterium stopu – wpływają na tempo zbieżności i czas trwania obliczeń. Integralną częścią zadania było również obliczenie promienia spektralnego macierzy iteracji, co umożliwiło weryfikację uzyskanych wyników eksperymentalnych z założeniami teoretycznymi.

3. Realizacja ćwiczenia

Pierwszym krokiem było wygenerowanie referencyjnego wektora rozwiązania $\bar{x}$ o rozmiarze $n \times 1$, którego elementy wylosowano ze zbioru $\{1, -1\}$. W celu zapewnienia powtarzalności wyników, ustawiono stałe ziarno generatora liczb pseudolosowych (`np.random.seed(47654678)`). Następnie wyznaczano wektor wyrazów wolnych:

$$b = A\bar{x}$$

Układ równań $Ax = b$ rozwiązywano iteracyjnie za pomocą metody Jacobiego. Wyznaczony w ten sposób wektor rozwiązań x był następnie porównywany z wektorem referencyjnym $\bar{x}$. Operację mnożenia macierzy przez wektor zaimplementowano za pomocą klasycznych pętli o złożoności obliczeniowej $O(n^2)$.

3.1. Implementacja metody Jacobiego

W celu wyznaczenia kolejnych przybliżeń wektora rozwiązań, macierz współczynników A poddano dekompozycji na część diagonalną D oraz resztową R , przyjmując zależność $A = D + R$. Przekształcenie to pozwala na zapisanie wyjściowego układu w postaci równoważnej $Dx = b - Rx$, co prowadzi bezpośrednio do ogólnego, macierzowego wzoru iteracyjnego metody Jacobiego:

$$x^{(k+1)} = D^{-1}(b - Rx^{(k)})$$

gdzie $x^{(k)}$ oznacza przybliżenie wektora rozwiązań uzyskane w k -tej iteracji. W programie operacja ta została zrealizowana komponentowo, kolejne wektory wynikowe były wyznaczane więc za pomocą wzoru:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1, j \neq i}^n a_{ij} x_j^{(k)} \right), \quad i = 1, \dots, n$$

3.2. Kryteria stopu

W trakcie wyznaczania rozwiązania badano liczbę iteracji oraz czas niezbędny do spełnienia zadanego kryterium stopu dla różnych wartości parametru zbieżności ρ . Wprowadzono limit maksymalnej liczby iteracji równy 10000. W obu przypadkach za miarę błędu przyjęto normę euklidesową, obliczaną za pomocą funkcji bibliotecznej `numpy.linalg.norm`:

$$\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$

3.2.1. Kryterium przyrostowe

Kryterium to przerywa obliczenia, gdy zmiana wektora rozwiązania pomiędzy dwiema kolejnymi iteracjami jest dostatecznie mała:

$$\|x^{(i+1)} - x^{(i)}\|_2 < \rho$$

gdzie $x^{(i)}$ – i -te przybliżenie wektora wynikowego, $i \in \mathbb{N}^+$, ρ – zadana dokładność.

3.2.2. Kryterium rezydualne

Kryterium rezydualne weryfikuje, jak dobrze aktualne przybliżenie spełnia wyjściowy układ równań:

$$\|Ax^{(i)} - b\|_2 < \rho$$

gdzie $x^{(i)}$ – i -te przybliżenie wektora wynikowego, $i \in \mathbb{N}^+$, ρ – zadana dokładność, A – macierz współczynników układu równań, b – wektor wyrazów wolnych.

3.3. Wektor początkowy

Aby zbadać wpływ punktu startowego na zbieżność algorytmu, przetestowano dwa warianty wektora x^0 o wymiarach $n \times 1$. Pierwszy z nich – wektor zerowy, stanowił punkt startowy bliski rzeczywistemu rozwiązaniu, ponieważ wektor referencyjny zawierał wyłącznie wartości -1 i 1 . Drugi wariant, w którym każda składowa przyjmowała losowo wyłącznie wartość -100 lub 100 , symulował punkt startowy daleki od rozwiązania. Pozwoliło to na ocenę stabilności i szybkości metody w trudniejszych warunkach.

3.4. Pomiar poprawności wyznaczenia rozwiązania

Pomiar poprawności algorytmu przeprowadzony został poprzez porównanie ostatecznie wyznaczonego wektora rozwiązań x z wektorem referencyjnym $\bar{x}$. Za metrykę błędu przyjęto maksymalny błąd bezwzględny:

$$\|x\|_{\max} = \max_{i \in \{1, \dots, n\}} (|x_i - \bar{x}_i|)$$

gdzie: x_i – i -ta składowa wyznaczonego wektora rozwiązań, $\bar{x}_i$ – i -ta składowa wektora referencyjnego

3.5. Promień spektralny macierzy i zbieżność

Warunkiem koniecznym i wystarczającym zbieżności metody iteracyjnej jest nierówność:

$$\rho_s(M) < 1$$

gdzie ρ_s jest promieniem spektralnym, czyli największą co do modułu wartością własną macierzy, a M to macierz iteracji. Dla metody Jacobiego macierz iteracji przyjmuje postać $M = -D^{-1}R$.

Promień spektralny ρ_s wyznaczono w programie jako maksymalną wartość bezwzględną ze zbioru wartości własnych macierzy M , wygenerowanych za pomocą bibliotecznej funkcji `numpy.linalg.eigvals`.

4. Analiza wyników

4.1. Promień spektralny

Przed przystąpieniem do analizy czasu i liczby iteracji, wyznaczono promień spektralny ρ_s macierzy iteracji dla wybranych rozmiarów układu n . Zebrane wyniki zestawiono w Tabeli 1.

Rozmiar układu n	Promień spektralny ρ_s
2	0.142857
3	0.227238
5	0.233644
10	0.236341
15	0.236891
20	0.237293
50	0.237888
100	0.238092
200	0.238195
300	0.238229
400	0.238246
500	0.238256
1000	0.238276
2000	0.238287
3000	0.238290
5000	0.238293

Tabela 1: Promień spektralny macierzy iteracji w zależności od rozmiaru układu

Wartość promienia spektralnego rośnie zauważalnie jedynie dla najmniejszych macierzy, po czym szybko dąży do asymptotycznego poziomu około 0.238. Uzyskane wyniki potwierdzają, że dla każdego badanego rozmiaru układu spełniony jest warunek zbieżności ($\rho_s < 1$), co w teorii gwarantuje skuteczność metody Jacobiego.

4.2. Czas działania, liczba iteracji oraz maksymalny błąd bezwzględny

W celu oceny efektywności oraz poprawności algorytmu Jacobiego, zestawiono ze sobą trzy kluczowe parametry: całkowity czas trwania obliczeń w sekundach, liczbę wymaganych iteracji do spełnienia warunku stopu oraz osiągnięty maksymalny błąd bezwzględny.

Badania przeprowadzono dla obu kryteriów stopu w zależności od rozmiaru układu N oraz tolerancji błędu ρ . Wyniki pomiarów dla wektora zerowego zestawiono odpowiednio w Tabela 3, Tabela 2 oraz Tabela 4, natomiast dane uzyskane dla wektora złożonego z wartości $-100, 100$ zaprezentowano w Tabela 6, Tabela 5 oraz Tabela 7.

4.2.1. Wektor zerowy

Rozmiar układu N	Kryterium przyrostowe				Kryterium rezydualne			
	10^{-3}	10^{-5}	10^{-9}	10^{-15}	10^{-3}	10^{-5}	10^{-9}	10^{-15}
2	5	8	12	19	5	8	12	20
3	6	10	16	25	6	10	16	25
5	6	10	16	26	7	10	17	27
10	7	10	16	26	7	10	16	10000
15	7	10	16	26	7	10	17	10000
20	7	10	16	26	7	11	17	10000
50	7	11	17	26	8	11	17	10000
100	8	11	17	27	8	11	17	10000
200	8	11	17	27	8	11	17	10000
500	8	11	17	31	9	12	18	10000

Tabela 2: Liczba iteracji dla wektora początkowego zerowego

Rozmiar układu N	Kryterium przyrostowe				Kryterium rezydualne			
	10^{-3}	10^{-5}	10^{-9}	10^{-15}	10^{-3}	10^{-5}	10^{-9}	10^{-15}
2	0.000025	0.000016	0.000022	0.000033	0.000017	0.000021	0.000031	0.000050
3	0.000015	0.000024	0.000039	0.000059	0.000023	0.000039	0.000062	0.000097
5	0.000031	0.000051	0.000072	0.000116	0.000063	0.000089	0.000136	0.000219
10	0.000098	0.000140	0.000222	0.000360	0.000185	0.000262	0.000421	0.281402
15	0.000219	0.000314	0.000490	0.000807	0.000416	0.000604	0.001018	0.597672
20	0.000381	0.000554	0.000859	0.001399	0.000724	0.001161	0.001776	1.042670
50	0.002306	0.003497	0.005463	0.008254	0.004970	0.006850	0.010618	6.195599
100	0.010071	0.013632	0.021088	0.033516	0.019449	0.026618	0.041110	24.256086
200	0.039848	0.054030	0.083704	0.131733	0.076894	0.107831	0.171458	96.837321
500	0.258272	0.356224	0.555835	0.997500	0.569745	0.765341	1.140350	642.273438

Tabela 3: Czas działania algorytmu [s] dla wektora początkowego zerowego

Rozmiar układu N	Kryterium przyrostowe				Kryterium rezydualne			
	10^{-3}	10^{-5}	10^{-9}	10^{-15}	10^{-3}	10^{-5}	10^{-9}	10^{-15}
2	5.95e-05	1.73e-07	7.22e-11	1.11e-16	5.95e-05	1.73e-07	7.22e-11	0.00e+00
3	7.95e-05	4.15e-07	5.76e-11	0.00e+00	7.95e-05	4.15e-07	5.76e-11	0.00e+00
5	1.60e-04	4.98e-07	1.09e-10	1.11e-16	3.58e-05	4.98e-07	2.37e-11	0.00e+00
10	4.62e-05	4.72e-07	6.91e-11	2.22e-16	4.62e-05	4.72e-07	6.91e-11	2.22e-16
15	6.03e-05	5.04e-07	1.15e-10	3.33e-16	6.03e-05	5.04e-07	2.33e-11	3.33e-16
20	4.46e-05	6.40e-07	5.25e-11	5.55e-16	4.46e-05	1.41e-07	1.57e-11	5.55e-16
50	5.66e-05	1.75e-07	1.98e-11	4.44e-16	9.79e-06	1.75e-07	1.98e-11	4.44e-16
100	1.16e-05	1.50e-07	2.10e-11	9.99e-16	1.16e-05	1.50e-07	2.10e-11	9.99e-16
200	9.70e-06	1.73e-07	2.12e-11	1.55e-15	9.70e-06	1.73e-07	2.12e-11	1.55e-15
500	9.07e-06	2.07e-07	2.29e-11	2.55e-15	3.50e-06	2.64e-08	3.65e-12	2.55e-15

Tabela 4: Maksymalny błąd bezwzględny dla wektora początkowego zerowego

4.2.2. Wektor o wartościach -100 i 100

Rozmiar układu N	Kryterium przyrostowe				Kryterium rezydualne			
	10^{-3}	10^{-5}	10^{-9}	10^{-15}	10^{-3}	10^{-5}	10^{-9}	10^{-15}
2	8	10	15	22	8	10	15	22
3	9	12	18	28	9	13	19	29
5	10	13	20	28	10	13	20	30
10	10	13	19	29	10	13	20	10000
15	10	13	19	29	11	14	20	10000
20	10	13	19	29	10	14	20	10000
50	11	14	20	29	11	14	20	10000
100	11	14	20	30	11	14	20	10000
200	11	14	20	33	11	14	21	10000
500	11	14	21	10000	12	15	21	10000

Tabela 5: Liczba iteracji dla wektora początkowego o wartościach -100 i 100

Rozmiar układu N	Kryterium przyrostowe				Kryterium rezydualne			
	10^{-3}	10^{-5}	10^{-9}	10^{-15}	10^{-3}	10^{-5}	10^{-9}	10^{-15}
2	0.000018	0.000017	0.000026	0.000038	0.000024	0.000026	0.000038	0.000060
3	0.000028	0.000032	0.000043	0.000077	0.000039	0.000057	0.000074	0.000125
5	0.000045	0.000058	0.000092	0.000131	0.000080	0.000105	0.000164	0.000241
10	0.000137	0.000184	0.000280	0.000434	0.000275	0.000340	0.000559	0.282892
15	0.000315	0.000411	0.000614	0.000904	0.000687	0.000853	0.001243	0.602898
20	0.000573	0.000763	0.001091	0.001602	0.001099	0.001472	0.002146	1.042325
50	0.003706	0.004519	0.006420	0.009217	0.006835	0.009233	0.012391	6.195766
100	0.013763	0.017496	0.025158	0.038005	0.026883	0.034234	0.048965	24.381810
200	0.053633	0.068363	0.097489	0.161964	0.105954	0.134337	0.201112	96.534110
500	0.355280	0.452060	0.675928	446.672336	0.764924	0.950588	1.332939	635.858765

Tabela 6: Czas działania algorytmu [s] dla wektora początkowego o wartościach -100 i 100

Rozmiar układu N	Kryterium przyrostowe				Kryterium rezydualne			
	10^{-3}	10^{-5}	10^{-9}	10^{-15}	10^{-3}	10^{-5}	10^{-9}	10^{-15}
2	1.75e-05	3.58e-07	2.13e-11	0.0	1.75e-05	3.58e-07	2.13e-11	0.00e+00
3	6.85e-05	9.09e-07	1.17e-10	2.22e-16	6.85e-05	1.81e-07	2.56e-11	0.00e+00
5	3.87e-05	4.93e-07	3.19e-11	2.22e-16	3.87e-05	4.93e-07	3.19e-11	0.00e+00
10	4.03e-05	6.1e-07	1.21e-10	2.22e-16	4.03e-05	6.1e-07	2.98e-11	2.22e-16
15	5.38e-05	1.01e-06	8.15e-11	2.22e-16	1.7e-05	1.8e-07	1.42e-11	2.22e-16
20	4.69e-05	8.11e-07	9.38e-11	2.22e-16	4.69e-05	1.7e-07	1.83e-11	2.22e-16
50	1.53e-05	1.98e-07	3.01e-11	7.77e-16	1.53e-05	1.98e-07	3.01e-11	7.77e-16
100	1.54e-05	2.15e-07	1.62e-11	1.11e-15	1.54e-05	2.15e-07	1.62e-11	9.99e-16
200	1.78e-05	1.19e-07	2.13e-11	1.55e-15	1.78e-05	1.19e-07	8.62e-12	1.55e-15
500	1.8e-05	1.2e-07	6.05e-12	3.33e-15	3.44e-06	4.47e-08	6.05e-12	3.33e-15

Tabela 7: Maksymalny błąd bezwzględny dla wektora początkowego o wartościach -100 i

100

4.3. Wnioski

Znaczenie wektora początkowego: W przeciwieństwie do wektora zerowego, inicjalizacja wartościami znacznie oddalonymi od rzeczywistego rozwiązania wymusza wykonanie większej liczby iteracji – zazwyczaj od 3 do 5 dodatkowych kroków w zależności od rozmiaru macierzy i zadanej tolerancji ρ . Przekłada się to na proporcjonalny wzrost całkowitego czasu obliczeń. Mimo tej różnicy warto zauważyć, że początkowe oddalenie od rozwiązania nie wpływa na rząd wielkości generowanego błędu bezwzględnego. Dowodzi to dużej stabilności badanej metody dla analizowanej macierzy. Algorytm potrzebuje nieco więcej pracy, aby stłumić duży błąd początkowy, jednak dzięki silnej zbieżności ostatecznie osiąga zadaną dokładność równie niezawodnie, co w przypadku startu z punktu położonego blisko celu.

Wpływ kryterium stopu: Pod względem liczby iteracji, dla umiarkowanych wartości tolerancji oba kryteria wykazują niemal identyczne tempo zbieżności. Mimo to pod względem czasu obliczeń, kryterium rezydualne wykonuje się średnio dwukrotnie wolniej niż przyrostowe, co jest bezpośrednim skutkiem konieczności dodatkowego mnożenia macierzy przez wektor w każdej iteracji, złożoność $O(N^2)$. W tym obszarze wartości tolerancji oba kryteria generują zbliżony, niski błąd bezwzględny. Zaobserwowano jednak, że warunek rezydualny jest znacznie bardziej wrażliwy na ograniczenia arytmetyki zmiennoprzecinkowej. Próba wymuszenia skrajnej dokładności $\rho = 10^{-15}$ prowadziła do utraty zbieżności i zatrzymania algorytmu dopiero przez narzucony limit 10000 iteracji, nie poprawiając przy tym realnie jakości samego rozwiązania. Zjawisko to jest skutkiem kumulacji błędów zaokrągleń typu float64 w trakcie każdorazowego mnożenia macierzy, co tworzy fizyczną barierę uniemożliwiającą wyzerowanie wektora rezyduum. Dla dalekiego punktu startowego problem ten dotknął również kryterium przyrostowe. Przy $N = 500$ i $\rho = 10^{-15}$ algorytm osiągnął limit 10000 iteracji, obrazuje to jak wydłużony proces dochodzenia do rozwiązania potęguje błędy zaokrągleń podwójnej precyzji. Wskutek tego algorytm ulega stagnacji, gdyż drobne błędy maszynowe stają się większe od wymaganej dokładności, skutecznie blokując ostateczne zbiegnięcie metody.

dłuższa ścieżka zbieżności sprzyja na tyle dużej kumulacji szumu numerycznego na granicy formatu float64, że algorytm traci zdolność do dalszej poprawy rozwiązania. co dowodzi, że duże oddalenie początkowe dodatkowo potęguje błędy numeryczne ujawniające się przy tak rygorystycznej tolerancji algorytmu.

Związek promienia spektralnego z tempem zbieżności: Analiza macierzy iteracji wykazała, że jej promień spektralny ρ_s jest zawsze ostro mniejszy od 1 i dla większych układów dąży do poziomu około 0.238. Powyższe zjawisko ma bezpośrednie przełożenie na wyniki numeryczne: liczba iteracji wymagana do osiągnięcia zadanej dokładności ulega jedynie minimalnemu zwiększeniu wraz ze wzrostem wymiaru macierzy N , wykazując wyraźną tendencję do stabilizacji.

Wpływ rozmiaru układu N : Całkowity czas działania algorytmu rośnie nieliniowo wraz ze wzrostem N , co empirycznie potwierdza złożoność kwadratową $O(N^2)$ metody Jacobiego. Liczba iteracji pozostaje stała, jednak koszt pojedynczej iteracji rośnie kwadratowo, co sprawia, że dla bardzo dużych układów metoda staje się kosztowna czasowo. Z kolei w kontekście osiągniętej precyzji, algorytm wykazuje znakomitą stabilność numeryczną – dla z góry ustalonej tolerancji błędu ρ , maksymalny błąd bezwzględny utrzymuje się na stałym poziomie, niezależnie od wymiaru układu.

Wpływ tolerancji błędu ρ : Zgodnie z oczekiwaniami, zaostrzanie tolerancji błędu, czyli zmniejszanie ρ skutkuje logarytmicznym wzrostem wymaganej liczby iteracji, co pociąga za sobą proporcjonalne wydłużenie całkowitego czasu działania algorytmu. Mniejsza tolerancja pozwala na znaczne zredukowanie maksymalnego błędu bezwzględnego, potwierdzając tym samym zbieżność metody do prawidłowego rozwiązania. Próba narzucenia skrajnej dokładności rzędu 10^{-15} skutkuje sprowadzeniem błędu do poziomu ograniczeń precyzji maszynowej typu float64, ok. 10^{-16} .

5. Podsumowanie

Przeprowadzone eksperymenty numeryczne potwierdzają wysoką skuteczność metody Jacobiego w przypadku badanej macierzy. Fundamentalne znaczenie ma promień spektralny macierzy ($\rho_s < 1$), który gwarantuje szybką zbieżność i czyni liczbę iteracji praktycznie niezależną od rozmiaru układu. Wydajność metody pozostaje jednak silnie uzależniona od zastosowanych parametrów. Zdecydowaną przewagę wykazuje kryterium przyrostowego – działa ono dwukrotnie szybciej i jest znacznie odporniejsze na błędy numeryczne niż kryterium rezydualne. O ile wybór wektora startowego ma znaczenie marginalne, kluczowym parametrem staje się tolerancja błędu ρ , która pozwala skutecznie zarządzać kompromisem między precyzją rozwiązania a nieliniowo rosnącym czasem obliczeń.